

DtDbScannerV2 — Double Top / Double Bottom Scanner — What It Does, Setting by Setting

File: nt8/indicators/DtDbScannerV2.cs **Status:** Discretionary alert tool. Backtested and mechanically rejected (S126 — see "Backtest history" below). Not currently on a live chart; DtDbScanner.cs (V1) is the version actually traded. V2 exists only as a resource-comparison clone that also fixes a hide/show bug — see "V2 vs V1" below.

1. What it is

A pattern-recognition indicator for tick charts. It watches the swing highs and lows the chart is already making and asks, continuously: do the last two highs (or lows) look like a double top (or double bottom)? When a candidate pair qualifies, it draws the structure, scores its quality 0–100, and fires an alert. It then tracks that pending structure bar by bar until price either confirms it (closes through the neckline), invalidates it (makes a new extreme beyond the pattern), or lets it expire (too many bars pass with no resolution).

It does **not** place trades. There is no order logic anywhere in the file — it is alerts + drawings only, execution is manual by design.

2. Core logic, step by step

2.1 Pivot detection (DetectPivotAt)

On every closed bar, it looks back SwingStrength bars and checks whether that bar is a strict swing high or low: its high (low) must be higher (lower) than every bar within SwingStrength bars on **both** sides. Because it needs bars on both sides, a pivot isn't confirmed until SwingStrength bars after it forms — this is the indicator's built-in detection lag. Confirmed highs and lows are appended to two rolling lists (highs, lows), each capped at 50 entries to bound memory.

2.2 Double Top (CheckDoubleTop) / Double Bottom (CheckDoubleBottom)

Every time a new pivot is added, it compares the **last two** pivots of that type (two highs for a top, two lows for a bottom) against four gates, all of which must pass:

1. **Peak symmetry** — the two highs (or lows) must be within PeakToleranceTicks of each other.
2. **Valley/peak depth** — the move between them (lowest low between the two highs, for a top; highest high between the two lows, for a bottom) must retrace at least MinValleyDepthTicks. This rejects a flat double top that never really pulled back.
3. **Spacing** — the two pivots must be between MinBarsBetween and MaxBarsBetween bars apart (too close = noise, too far = unrelated structures).
4. If all three pass, it computes a **quality score** (below) and opens a pending structure — essentially "a DT/DB has formed, now watching for confirmation."

Only one pending top and one pending bottom can exist at a time; a new qualifying pair overwrites the previous pending one of the same type.

2.3 Quality score (ScoreStructure, 0–100)

A weighted blend of four factors, each normalized to 0–1:

Weight	Factor	What it rewards
35%	Symmetry	the two peaks/troughs being closer together in price (1.0 at identical, 0 at the tolerance edge)
25%	Depth (ATR-relative)	the pullback/rally between the two pivots sitting in a "sane" band of 1.5–3× ATR(14) — too shallow or too deep both score lower (trapezoid function Band)
15%	Time symmetry	the two legs (first pivot→mid, mid→second pivot) taking roughly equal bar counts
25%	Trend context	there being a real prior move into the first peak/trough (a rally before a top, a decline before a bottom), sized relative to 3× ATR

This score is what gates alerts (MinScoreToAlert) and is stamped on the chart label/alert text. Nothing currently ranks across symbols/timeframes with it — the code has a stub comment ("RANKER: register score here") where that would plug in; it isn't implemented.

2.4 Pending-structure management (ManagePending, runs every bar)

Each pending top/bottom is resolved one of three ways:

- **Invalidated** — price makes a new high beyond the pattern (for a top) or new low beyond it (for a bottom) before confirming. The setup is discarded silently, no alert.
- **Expired** — ConfirmWithinBars bars pass since the structure formed with no confirmation. Discarded silently.
- **Confirmed** — the bar closes through the neckline (the valley/peak between the two pivots) by at least NecklineBufferTicks. Fires a high-priority alert, draws a confirm arrow, and clears the pending structure.

2.5 Drawing

If ShowDrawings is on, a forming structure gets two trend lines (pivot→neckline→pivot), a dashed neckline, and a text label with the score. A confirm fires an up/down arrow. All of these are plain Draw.NinjaScript objects — V2's only functional addition over V1 is making their .IsVisible follow the indicator's own show/hide toggle (see below).

2.6 Alerts

Two alert types, both via NT8's native Alert() (so they respect the user's configured alert sound/popup/log) plus a Print() echo to the NinjaScript Output tab:

- "... DT/DB forming score NN" — only if score $\geq$ MinScoreToAlert.
- "... DT/DB CONFIRMED score NN neckline PRICE" — always fires regardless of score.

3. V2 vs V1 — why two files exist

DtDbScanner.cs (V1) and DtDbScannerV2.cs are **byte-identical in detection/scoring logic**. The only difference: V1's Draw objects aren't tied to the indicator's own IsVisible flag at all (an NT8 platform quirk — IsVisible only gates an indicator's own OnRender, and V1 draws everything via Draw.Line/Draw.Text, which NT8 treats as independent chart objects with their own lifetime). Toggling V1 off in the Indicator Manager did nothing visually.

V2 fixes this by keeping every Draw return value in a list (_drawObjs) and running a 250ms DispatcherTimer that polls for an IsVisible flip and pushes it onto every tracked object, independent of bar flow (a bar-close-only sync was tried first and rejected — it can sit stale indefinitely between bars). This was verified working (user F5'd and confirmed) and committed, but **V2 is currently an idle parallel copy** — V1 remains the one actually on charts, kept this way intentionally in case the original needs a future logic update. Whether a future change should go into V1 again or make V2 the new base is an open question, not yet decided.

4. Backtest history (why this is discretionary-only)

A full walk-forward study (scripts/dtddb_walkforward.py, full RTH history 2021-06-18 to 2026-10-01, calendar-year expanding folds, ES-only, ≥ 3 trades/day gate, stop/target sweep layered on a 6,480-combo structure/confirm grid) found **no bar type survives**: only 1500t and 2000t bars ever clear the frequency gate, and both lose money out-of-sample (1500t: PF 0.92, -\$96,850 over 3,448 trades; 2000t: PF 0.93, -\$45,100 over 2,263 trades). This thread is closed — the indicator is kept purely as a manual/discretionary alerting tool, not a mechanical signal.

5. Settings reference

All properties are visible on the indicator's parameters panel in NT8, grouped as shown.

Group 1 — Structure (what counts as a double top/bottom)

Setting	Default	What it controls	Effect of raising it	Effect of lowering it
Swing strength	4	Bars required on each side of a bar to confirm it as a pivot	Fewer, stronger/more reliable swings; more detection lag (pivot not confirmed until this many bars later)	More pivots, noisier, less lag
Peak tolerance (ticks)	6	Max allowed price difference between the two peaks (or two troughs)	More pairs qualify as "equal enough"; looser pattern definition	Only near-identical peaks qualify; fewer, stricter matches
Min valley depth (ticks)	10	Minimum pullback/rally required between the two pivots	Rejects shallower, weaker-looking patterns	Allows flatter, less-defined patterns through
Min bars between	6	Minimum bar spacing between the two pivots	Filters out pivots that are too close together (noise)	Allows tighter, faster patterns
Max bars between	120	Maximum bar spacing between the two pivots	Allows slower-forming, more spread-out patterns	Restricts to faster-forming patterns only

Group 2 — Confirmation (how a pending structure resolves)

Setting	Default	What it controls	Effect of raising it	Effect of lowering it
Neckline buffer (ticks)	2	How far past the neckline the close must break to count as confirmed	Fewer false confirms, but confirmation comes later/less often	Faster confirms, more prone to false breaks
Confirm within bars	150	How long a pending structure is kept alive waiting for confirmation	Patterns stay "live" longer before expiring unconfirmed	Patterns expire (get dropped) sooner if price doesn't resolve them

Group 3 — Alerts

Setting	Default	What it controls	Effect of raising it	Effect of lowering it
Min score to alert	50	Minimum quality score (0–100) required to fire a "forming" alert	Only the highest-quality setups alert; quieter	More setups alert, including weaker ones. (Note: the "CONFIRMED" alert always fires regardless of this setting — it only gates the "forming" alert.)
Alert re-arm (sec)	30	Re-arm window passed to NT8's native Alert(), i.e. minimum time before the same alert ID can re-fire	Less repeat-alert spam	More frequent repeat alerts possible
Show drawings	true (on)	Whether the structure lines/neckline/score label/confirm arrows are drawn on the chart at all	—	Turning off suppresses all visuals; alerts/Print output still fire

6. Practical notes

- Designed for **tick charts** specifically — it reads BarsPeriod.Value as the tick interval for self-labeling alerts, and prints a warning (but still runs) if the chart isn't a tick chart.
- Calculate.OnBarClose is hardcoded — detection only happens on closed bars, which avoids repaint but means confirmation/invalidation can only trigger at bar close, not intra-bar. (Code comment notes OnEachTick would give faster neckline-break detection if ever revisited.)
- Needs at least 2 SwingStrength + 2 bars of history before it starts detecting anything.